Team: Traffic Light management System

Ansh Singh, Paras Arora, Pratyaksh Chanda, Shreshth Mehrotra

PROJECT DESCRIPTION

This project deals with the problem of mismanaged traffic.
It works by intelligently controlling the time
of red and green signal by calculating the traffic density at any particular signal. More the traffic density, more is the green signal time and likewise, lesser the traffic density, lesser the time. Overall, this saves time, fuel and avoids traffic hustle.

WORKING

This traffic system will work using YOLO (You Look Only Once) model as follows (considering real scenario)-:

1)A snapshot/clip of traffic by CCTV cameras will be taken.

2)Then this snapshot/clip will be passed to computers for object detection by using YOLO model. The various vehicles will be

segregated into classes such as “car”, “bus” etc, and the traffic density can be calculated.

3)Then, the ***scheduling algorithm*** i.e., the algorithm controlling the time of green, yellow and red signal will accordingly adjust the time.

4) The scheduling/signal switching algorithm takes into account the following factors:

- The number of lanes
- Lag suffered by each vehicle during start-up
- The ***nonlinear*** increase in lag suffered by the vehicles which are at the back
- The maximum and minimum green signal time that can be set, so that there is no “starvation” in lanes with no traffic.

RESOURCES, TOOLS AND APPLICATIONS INVOLVED

- **YOLO algorithm:** YOLO is a state of the art, modified CNN (Convolutional Neural Network) architecture used for object detection. (This is the meat of our project)

  The following will be used for training/implementing the YOLO algorithm:

- **Pytorch:** PyTorch is an open-source machine learning library based on Primarily python, and C++ along with cuda. It is used for creating deep learning models.
- **Keras:** Keras is an open-source software library that provides a Python interface for artificial neural networks.
- **TensorFlow** :A free and open-source software library for machine learning

Resources used:

- **Deep learning for vision systems: Mohammed engeldy :** chapters: 1,2,3,7

- **Towards Data science:** https://towardsdatascience.com/

- **freecodecamp.org:** https://www.youtube.com/channel/UC8butISFwT-Wl7EV0hUK0BQ

DETAILS OF IMPLEMENTATION

1)Frequency of snapshots:

Snapshots of the next signal are taken when the current signal shows 5 seconds on the yellow timer. (The processing time of the signal algorithm is 5 secs). As 5 seconds elapse, the current signal becomes red, and the next signal becomes green, with the time set by the algorithm.

2)Signal switching

$$GST = \frac{\sum_{vehicleClass} (NoOfVehicles_{vehicleClass} * AverageTime_{vehicleClass})}{(NoOfLanes + 1)}$$

where:

- GST is green signal time
- noOfVehiclesOfClass is the number of vehicles of each class of vehicle at the signal as detected by the vehicle detection module,
- averageTimeOfClass is the average time the vehicles of that class take to cross an intersection, and
- noOfLanes is the number of lanes at the intersection.

3)Signal switching process

- By default, the algorithm gives 20 seconds to the traffic to pass the first lane, and all the other lanes are stopped (i.e. red signal)

- When the green signal timer of the first lane reaches 5 seconds, the red signal time of the next lane is set to 10 seconds.

- As the green signal time reaches 0 seconds, (and the next signal is now 5), the green signal switches to yellow, and is set to 5 seconds.

- Now the first signal is yellow, and has a time of 5 seconds, and the next signal is red, and it also has a time of 5 seconds.

- At this instant, a snapshot of the next lane is taken, the number of objects of each vehicle class is calculated and is passed to the algorithm for processing the green signal time required.

- Now, as 5 seconds pass, the first signal becomes red, and the next signal becomes green, with the time set as per the algorithm.

- This process is repeated for all the subsequent lanes.

4)dl Framework

TensorFlow and keras together.

5)Time required:

10-12 days

YOLOV5 MODEL WORKING

1) Setup:

We had used **KITTI** dataset to train our model.

https://www.kaggle.com/twaldo/kitti-object-detection

This dataset contains 7500 images and we have used 1600 for validation. We have used following 9 classes to define an image :-

['Car', 'Bus', 'Truck', 'Mortorcycle', 'Person_sitting', 'Cyclist', 'Tram', 'Misc', 'DontCare']

Each image in this dataset contains labels x,y,w and

H, where they all are normalised i.e. between 0 and 1.

We first extracted (x_c, y_c) coordinates of the centre of the bounding box from the label file in the KITTI dataset. The label file of the KITTI dataset had (right, top, left, bottom) coordinates of the bounding boxes. We found the coordinates of the centre by (r+l/2), (t+b/2). Next, we found width and height of the bounding box by w_b = r-l and h_b = t- b.

Then we normalised by dividing x_c and w_b by W: width of the **image** and by dividing y_c and h_b by H: height of the **image**.

The label file for each image is of the format [I, x, y, w, h]:-

- Where I is the class index (starting from 0), e.g 0 for car, 2 for Truck etc.
- And, x, y, w and h are normalised parameters as discussed.

2) **Training :-** we used **train.py** python file which made use of argparse, numpy, and pytorch to train the model. We made use of the following command to initiate training of our YOlO model:

```
$ python train.py --img 640 --batch 4 --
epochs 5 --data kittid.yaml --weights
yolov5s.pt
```

which means that we trained for 5 epochs with batch size 4, image size 640 and the data was accessed through kiitid.yaml, which contained the paths for training and validation folders and other information like number of classes and class list.

3)Detection:- for detection we used detect.py file. We made us of argparse, numpy, pytorch, tensorflow, and open cv for image detection and counting numbers of cars, buses, etc..

INTERGRATING WITH PYTHON SIGNAL ALGORITHM

We now integrate the detect.py file along with our python code for signal switching process and calculating the green time.

The detect () function was called with parameters of the image, for e.g path, size, and also parameters of the model, for e.g, the weights and biases file, to process the snapshot and calculate the number of car, buses, etc. This data is then stored in a list and supplied to gst () function which calculates the green time from the formula.

$$GST = \frac{\sum_{vehicleClass} (NoOfVehicles_{vehicleClass} * AverageTime_{vehicleClass})}{(NoOfLanes + 1)}$$

where:

- GST is green signal time
- noOfVehiclesOfClass is the number of vehicles of each class of vehicle at the signal as detected by the vehicle detection module,
- averageTimeOfClass is the average time the vehicles of that class take to cross an intersection, and
- noOfLanes is the number of lanes at the intersection.

Number of lanes for our implementation is 4.

Now, the list containing times for (RYG) for each signal is accordingly updated and the countdown () function is called again.

WORKING MODEL (SIMULATION)

We have used tinkercad to simulate this process on an Arduino UNO board.

(link: https://www.tinkercad.com/things/i1Ndch9GYBg)

The gst () function is used to calculate the green time which is then used for controlling the delay () for the green LEDS.

The time left on the green signal is displayed by an LCD(which is controlled by a separate Arduino), which received its data via serial communication with the Arduino that controls the lights.

